

Introduzione

Compiti di un sistema operativo

Il **sistema operativo** è un programma che controlla altri programmi ed avente un'interfaccia tra applicazione e hardware.

Gli obiettivi principali del SO sono convenienza, efficienza e capacità di evolversi.

Sistema di elaborazione

Il sistema di elaborazione è costituito da **software** e **hardware**, organizzati su più livelli gerarchici e collegati da interfacce.



Hardware

1. Execution Hardware

- Unità di esecuzione della **CPU**.
- Esegue le istruzioni secondo l'ISA.

2. Memory Translation

- Gestisce la traduzione degli indirizzi logici in fisici (es. MMU).

3. System Interconnect (bus)

- Collega CPU, memoria, dispositivi I/O.

4. Main Memory

- Memoria principale (**RAM**).

5. I/O Devices and Networking

- Dispositivi di input/output e rete.



Software

1. Application Programs

- Programmi usati dagli utenti finali.
- Interagiscono con l'API (Application Programming Interface).

2. Libraries/Utilities

- Raccolte di funzioni e strumenti di supporto.
- Forniscono un'interfaccia binaria (ABI) alle applicazioni.

3. Operating System

- Gestisce risorse hardware e software.
- Si appoggia all'**Instruction Set Architecture (ISA)** per dialogare con l'hardware.

Interfacce Principali

- **ISA (Instruction Set Architecture)**: interfaccia tra OS e hardware.
- **API (Application Programming Interface)**: collega applicazioni e librerie/OS.
- **ABI (Application Binary Interface)**: collega codice compilato alle librerie e al sistema operativo.

Il SO è un normale programma eseguito dal processore e dipende da esso.

Evoluzione dei sistemi operativi

Un SO deve evolversi per aggiornarsi, supportare nuovi tipi di hardware e soddisfare le richieste di nuovi servizi.

Prima generazione ('45-'55)

Non c'era ancora un SO ma il controllo avveniva da console, tutto gestito da valvole.

I tempi di preparazione erano lunghi e c'erano problemi di schedulazione.

Seconda generazione ('55-'65)

Sistemi a lotti: primi transistor e sistemi gestiti da un operatore, con lavori raggruppati in lotti sul dispositivo di ingresso.

Il **monitor** ordina il caricamento dei programmi e al termine del lavoro il controllo torna al monitor.

Furono introdotte **nuove funzioni hardware** con:

- **Protezione della memoria:** evita che un processo acceda alla memoria di un altro.
- **Timer:** consente al sistema operativo di riprendere il controllo dopo un certo periodo.
- **Istruzioni privilegiate:** solo il sistema operativo può eseguire certe istruzioni critiche.
- **Interruzioni:** meccanismo per rispondere a eventi hardware/software.

Overhead

Indica le risorse accessorie, **richieste in sovrappiù** rispetto a quelle strettamente necessarie per ottenere un determinato scopo in seguito all'introduzione di un metodo o di un processo più evoluto più generale.

Parte della memoria è riservata al sistema operativo (monitor) e parte del tempo della CPU è spesa per il monitor stesso.

Vanno pensati dei monitor che non sprechino spazio e tempo.

Terza generazione ('65-80)

Circuiti integrati: una piastrina svolge il lavoro di centinaia di transistor.

Multiprogrammazione

L'uso di circuiti integrati ha consentito l'introduzione della **multiprogrammazione**, che permette a **più programmi di essere caricati in memoria contemporaneamente**, migliorando l'uso della CPU.

Ogni programma utilizza il processore per un suo calcolo, una frazione minima di tempo, quindi l'idea è di usarlo anche nelle pause.

Lettura di un record → esecuzione di X istruzioni → Scrittura di un record

La percentuale di utilizzo della CPU è $1/\text{tempo totale}$.

Nella multiprogrammazione, se la memoria è sufficiente per eseguire dei processi, si fanno partire tutti insieme dal momento che il processore è disponibile; in seguito si scambiano in base a quando devono accedere a memoria o CPU.

Se devono accedere contemporaneamente si mettono

in attesa in base alla priorità.

I problemi che nascono sono:

1. Gestione della memoria più complessa.

2. Scheduling del processore: se entrambi i processi hanno bisogno del processore, uno deve aspettare.

Le istruzioni al SO sono impartite tramite **JCL (Job Control Language)**.

Time sharing

Il **processore viene condiviso da più utenti**: ogni utente ha il proprio terminale con cui accede alla macchina per il tempo assegnato.

Il S.O. alterna l'esecuzione di ciascun programma utente per un periodo (**quanto di elaborazione**).

Essendoci tanti terminali chiunque può accedere alla macchina, quindi bisogna verificare che abbia i permessi di accedere e di usare solo **funzioni specifiche**, con un **tempo di uso limitato**.

- Uno spazio di memoria viene assegnata al S.O.
- Carica un lavoro e lo esegue.
- Viene sospeso.
- La memoria si libera.
- Viene caricato ed eseguito il secondo lavoro.

L'obiettivo del time sharing è minimizzare il tempo di risposta, con **comandi inseriti da terminale**.

Quarta Generazione ('80-oggi)

Vengono creati **LSI (Integrazione su scala)**: circuiti integrati con più transistor.

Ognuno ha il suo calcolatore (**Personal Computer**) con tempo di risposta migliorata e interfaccia migliore.

Quinta Generazione ('90-oggi)

Avvento degli **smartphone** con energia limitata, quindi bisogna tener conto dell'energia usata dai vari programmi per non farli scaricare.

Quantità enorme di nuovi dispositivi (geolocalizzazione...) con cui il sistema operativo deve interfacciarsi.

Sistema di Elaborazione

Componenti del Sistema di Elaborazione

Un sistema di elaborazione è costituito da diversi componenti interconnessi:

- **CPU (Unità Centrale di Elaborazione):** esegue le istruzioni.
- **Memoria Principale:** memorizza dati e istruzioni in esecuzione.
- **Modulo I/O:** gestisce le operazioni di input e output.
- **Bus di Sistema:** connette CPU, memoria e dispositivi I/O.
- **Registri CPU:**
 - **PC (Program Counter):** memorizza l'indirizzo della prossima istruzione.
 - **IR (Instruction Register):** contiene l'istruzione in esecuzione.
 - **MAR (Memory Address Register):** indirizza la memoria.
 - **MBR (Memory Buffer Register):** memorizza dati temporaneamente.
 - **I/O AR e I/O BR:** gestiscono l'interazione con dispositivi di input/output.

Unità Centrale e Tipologie di Processori

Microprocessore

- Un solo processore su un chip.
- **Multiprocessore:** più core in un singolo chip (**socket**).
- **Cache** per velocizzare l'accesso ai dati.

GPU (Graphics Processing Unit)

Utilizza la tecnica **SIMD (Single-Instruction Multiple-Data)** → elevato numero di processori identici eseguono la stessa sequenza di istruzioni su insiemi diversi di dati.

Ottimizzata per elaborazioni parallele, non solo per grafica.

DSP (Digital Signal Processor)

- Progettato per elaborare segnali audio e video.
- Supporta crittografia e compressione dati.

SoC (System on Chip)

- Include CPU, memoria e periferiche in un unico chip, usato in dispositivi mobili.

Ciclo Fetch-Execute

Il processore esegue istruzioni secondo un ciclo:

1. **Fetch:** recupera l'istruzione dalla memoria.
2. **Decode:** decodifica l'istruzione.

3. **Execute:** esegue l'operazione.
4. **Store:** memorizza il risultato.

Tipi di istruzioni:

- **Processore-Memoria:** accesso ai dati in memoria.
- **Processore-I/O:** comunicazione con dispositivi esterni.
- **Elaborazione dati:** operazioni logiche e matematiche.
- **Controllo:** alterano il flusso del programma (es. salti condizionali).

Interruzioni

Le interruzioni migliorano l'efficienza della CPU, permettendole di eseguire altre operazioni mentre attende eventi esterni.

Tipologie di interruzioni

- **Di programma:** errori di esecuzione (overflow, accessi a indirizzi non validi).
- **Di timer:** permettono al sistema operativo di controllare il tempo di esecuzione dei processi.
- **Di I/O:** segnalano il completamento di un'operazione di input/output.
- **Hardware:** guasti o malfunzionamenti.

Ciclo Fetch-Execute con Interruzioni

1. L'istruzione viene recuperata ed eseguita.
2. Il sistema controlla se è avvenuta un'interruzione.
3. Se presente, attiva il gestore di interruzioni.
4. Una volta completata l'interruzione, l'esecuzione riprende.

Gestione delle Interruzioni

1. Il dispositivo I/O o hardware invia un segnale di interruzione.
2. Il processore completa l'istruzione corrente.
3. Il processore salva lo stato attuale (PSW e PC) nello stack.
4. Viene eseguita la routine di gestione dell'interruzione.
5. Lo stato originale viene ripristinato e il programma riprende.

Interruzioni Multiple

Due approcci per gestire più interruzioni simultanee:

1. **Disabilitare le interruzioni** mentre se ne gestisce una.
2. **Assegnare priorità** alle interruzioni per eseguire prima quelle più urgenti.

Architettura dei Sistemi Operativi

Aspetti principali nello sviluppo di un OS

- **Processi:** gestione dell'esecuzione di più programmi.
- **Gestione della memoria:** allocazione efficiente della memoria.
- **Protezione e sicurezza:** protezione dei dati e delle risorse.
- **Schedulazione e gestione delle risorse:** distribuzione equa della CPU e della memoria.
- **Struttura del sistema:** organizzazione e modularità del codice.

Approcci progettuali

- **Microkernel:** separa le funzioni di base in moduli indipendenti.
- **Multithreading:** consente l'esecuzione concorrente di più thread all'interno di un processo.
- **Multiprocessing simmetrico:** distribuisce equamente il carico di lavoro su più CPU.
- **Sistemi operativi distribuiti:** più macchine lavorano insieme come un unico sistema.
- **Architettura orientata agli oggetti:** migliora la modularità e la manutenibilità.

Macchine Virtuali

- Un **sistema operativo può creare macchine virtuali**, ciascuna con il proprio OS e memoria.
 - Un **Virtual Machine Monitor (VMM)** gestisce la comunicazione tra le macchine virtuali e l'hardware.
-

Struttura del S.O.

Struttura del Sistema Operativo

Un sistema operativo (OS) è composto da diversi livelli e fornisce un'**interfaccia tra l'hardware e i programmi utente**. Le sue funzioni principali includono l'esecuzione di programmi, la gestione delle operazioni di I/O, il controllo dei file system e la comunicazione tra processi.

Servizi per l'Utente

Il sistema operativo mette a disposizione dell'utente diversi servizi:

- **Interfaccia Utente:** Può essere una CLI (Command Line Interface), una GUI (Graphical User Interface) o un sistema batch.
- **Esecuzione dei Programmi:** Gestisce il caricamento, l'esecuzione e la terminazione dei programmi.
- **Operazioni di I/O:** Consente la gestione dei dispositivi di input e output.
- **Gestione dei File:** Permette la creazione, cancellazione, lettura, scrittura e gestione dei permessi.
- **Comunicazione tra Processi:** Può avvenire tramite memoria condivisa o scambio di messaggi.
- **Rilevazione degli Errori:** Controlla possibili errori hardware, di I/O o nei programmi utente.

Funzioni per l'Efficienza del Sistema

Il sistema operativo deve ottimizzare l'uso delle risorse disponibili:

- **Allocazione delle Risorse:** Controlla e gestisce l'uso del processore, della memoria, del disco e degli altri dispositivi.
- **Accounting:** Tiene traccia dell'utilizzo delle risorse.
- **Protezione e Sicurezza:** La protezione garantisce il corretto uso delle risorse, mentre la sicurezza regola gli accessi al sistema.

Interprete dei Comandi (CLI) e Interfaccia Grafica (GUI)

- La **CLI** esegue comandi testuali specificati dall'utente e può essere implementata nel kernel o come programma separato, ideale per compiti ripetitivi (*shell scripts*).
- La **GUI** offre un'interfaccia più intuitiva con finestre, icone e menu, ma non sempre consente l'accesso a tutte le funzioni.

Chiamate di Sistema

Le chiamate di sistema sono il principale mezzo di **interazione** tra i **programmi e il sistema operativo**. Sono accessibili tramite **API** e gestite dalla libreria di supporto run-time. I parametri possono essere passati tramite registri, tabelle di memoria o nello stack.

Categorie di Chiamate di Sistema

- **Controllo dei Processi:** Creazione, esecuzione, terminazione e gestione della memoria.
- **Gestione dei File:** Operazioni sui file come apertura, chiusura, lettura e scrittura.
- **Gestione dei Dispositivi:** Interazione con dispositivi hardware.

- **Gestione delle Informazioni:** Configurazione di data, ora e attributi di file e processi.
- **Comunicazione:** Trasmissione di messaggi e accesso a memoria condivisa.
- **Protezione:** Controllo degli accessi e gestione dei permessi.

Programmi di Sistema

I programmi di sistema forniscono strumenti per lo **sviluppo e l'esecuzione di software**, tra cui:

- Gestione dei file.
- Strumenti di modifica dei file (**editori** di testo).
- Utility per il supporto alla programmazione.
- Servizi di comunicazione e background.

Progettazione del Sistema Operativo

- **Obiettivi e Specifiche:** Il sistema operativo deve bilanciare esigenze dell'hardware, del tipo di sistema e degli utenti.
- **Implementazione:** Oggi si usa prevalentemente il C e C++, con alcune parti critiche scritte in Assembly.

Strutture del Sistema Operativo

- **Sistemi Monolitici:** Non modulari, tutto il codice è integrato in un unico blocco (*MS-DOS, UNIX*).
- **Sistemi Stratificati:** Strutturati a livelli, dove ogni livello fornisce servizi al successivo, tuttavia generando overhead.
- **Microkernel:** Sposta il più possibile le funzionalità ai programmi utente (server) con comunicazione tramite messaggi, semplificando il debug e l'aggiornamento ma generando overhead.
- **Moduli:** Approccio modulare che consente il caricamento dinamico di funzionalità, il nucleo contiene solo le informazioni fondamentali.

Debugging

Il debugging del sistema operativo aiuta a **identificare errori** e problemi di prestazione attraverso log e strumenti di monitoraggio.

Generazione del Sistema Operativo

1. **Scrivere** o ottenere il codice sorgente
2. **Compilare** il SO
3. **Configurare** il SO per il sistema su cui verrà eseguito
4. **Installare** il SO.
5. **Avviare** il PC ed il nuovo SO.

La **configurazione** può realizzarsi con **modifiche del codice** prima della compilazione, selezione di moduli **precompilati** o con un sistema guidato da **tabelle**.

Avvio del sistema

L'esecuzione inizia con il **bootstrap loader** presente nella ROM, che carica il kernel in memoria e avvia il sistema.

Processi

Definizione di Processo

Un **processo** (o *job*) è un programma in esecuzione. A differenza di un semplice file eseguibile (che è un'entità passiva), un processo è un'entità attiva che dispone di un program counter (PC) e di risorse associate.

Elementi del processo

- **Program counter e registri:** Contengono l'immagine del processore.
- **Stack:** Memorizza dati temporanei come variabili locali, indirizzi di ritorno e parametri delle funzioni.
- **Sezione testo:** Contiene il codice del programma.
- **Sezione dati:** Contiene variabili globali.
- **Heap:** Area per l'allocazione dinamica della memoria.

Un programma può essere associato a più processi, ognuno con il proprio stato e risorse.

Stati del Processo

Un processo può trovarsi in diversi stati:

- **Nuovo:** Il processo è in fase di creazione.
- **In esecuzione:** Sta attivamente utilizzando la CPU.
- **In attesa:** È sospeso in attesa di un evento (es. I/O).
- **Pronto:** È in attesa di essere eseguito dalla CPU.
- **Terminato:** Ha completato la sua esecuzione.

Il passaggio tra stati avviene tramite lo **scheduler** e la gestione delle risorse.

PCB (Process Control Block)

Ogni processo è associato a una struttura dati chiamata **PCB**, che contiene:

- **Stato** del processo
- **Program counter**
- **Registri** della CPU
- Parametri per lo **scheduling**
- Informazioni sulla **memoria**
- Informazioni sull'**I/O**
- informazioni per l'**accounting**

Threads

Un processo può avere più **thread**, che permettono l'esecuzione parallela su più core. Il PCB include informazioni su ciascun thread.

Code di Schedulazione

I PCB vengono inseriti in diverse code:

- **Coda dei job:** Contiene tutti i processi.
- **Coda dei ready:** Contiene i processi pronti per l'esecuzione.
- **Code dei dispositivi:** Contengono i processi in attesa di un dispositivo specifico.

Schedulatori

Gli schedulatori gestiscono l'assegnazione della CPU:

- **Schedulatore a breve termine (CPU scheduler):** Seleziona quale processo pronto eseguire. Deve essere veloce ed eseguito con grande frequenza.
- **Schedulatore a lungo termine (Job scheduler):** Controlla quali processi inserire nella coda dei pronti e controlla la multiprogrammazione. Applica politiche complesse e non è eseguito con grande frequenza.
- **Schedulatore a medio termine:** Gestisce lo **swapping**, ovvero il caricamento e lo scaricamento di processi dalla memoria principale.

Processi CPU-bound vs I/O-bound:

- **CPU-bound:** Utilizzano principalmente la CPU.
- **I/O-bound:** Attendere frequentemente operazioni di input/output.

Con queste due distinzioni la schedulazione realizza delle politiche per l'assegnazione della CPU.

Il cambio di contesto introduce un overhead che dipende da fattori come la velocità della memoria e il numero di registri.

Creazione e Terminazione dei Processi

Creazione

Un processo può creare **processi figli**, formando un **albero dei processi**, in cui ogni processo è identificato da un **PID** (Process ID).

Risorse

- Il processo figlio può **ereditare risorse** dal padre (ripartite o condivise) o dal S.O.
- Il figlio può essere un duplicato del padre o caricare un nuovo programma.
- Il genitore e il figlio possono eseguire in parallelo oppure il genitore può attendere la terminazione del figlio.

Esempio in Linux:

```
#include <sys/types.h>
#include <stdio.h>
#include <unistd.h>
int main() {
    pid_t pid;
    /* fork a child process */
    pid = fork();
```

```

if (pid < 0) {
    fprintf(stderr, "Fork Failed");
    return 1;
}
/* child process */
else if (pid == 0) {
    execlp("/bin/ls", "ls", NULL);
}
else {
    /* parent will wait for the child to complete */
    wait(NULL);
    printf("Child Complete");
}
return 0;
}

```

Terminazione

- Un processo termina con la chiamata di sistema `exit()` e il sistema rilascia le risorse, che vengono deallocate.
- Il processo genitore può terminare un figlio con `abort()`, ad esempio per eccesso di risorse o compito non più necessario.

Dopo il `wait()` del genitore possono crearsi processi:

- **Zombie:** Un processo terminato ma il cui PCB non è stato ancora rimosso.
- **Orfani:** Processi il cui genitore è terminato prima di loro.

Threads

Processi e Threads

I processi nei sistemi operativi sono caratterizzati da due aspetti fondamentali:

- **Risorse allocate:** spazio di indirizzamento virtuale, RAM, dispositivi I/O e file.
- **Scheduling:** gestione dell'esecuzione alternata dei processi sulla CPU.

Tuttavia, è possibile separare la gestione delle risorse dalla gestione dell'esecuzione:

- **Processo (o task):** possiede le risorse.
- **Thread (o processo leggero):** è l'unità che viene assegnata alla CPU per l'esecuzione.

Multithreading

Il **multithreading** è la capacità di un sistema operativo di gestire più tracce di esecuzione concorrenti all'interno di un singolo processo.

Si possono individuare quattro scenari principali:

1. **Single-threading:** un processo, un solo thread (es. MS-DOS).
2. **Multi-threading a livello di processo:** un processo con più thread (es. Java runtime).
3. **Multi-processo con un thread ciascuno:** più processi indipendenti con un solo thread (es. UNIX tradizionale).
4. **Multi-processo con più thread per processo:** ogni processo ha più thread (es. Windows, Solaris, UNIX).

Il Processo

In un ambiente multi-thread, il processo è l'unità di allocazione delle risorse ed un'unità di protezione, avente:

- **Spazio di indirizzamento virtuale** contenente l'immagine del processo.
- Accesso **protetto** al processore o ad altri processi.
- Uno o più **thread**.

Thread: Definizione e Componenti

Un thread è caratterizzato da:

- **Stato:** pronto, in attesa o in esecuzione.
- **Contesto del thread:** immagine del processore, salvata quando il thread non è in esecuzione.
- **Stack di esecuzione:** memorizza le variabili locali e gli indirizzi di ritorno.
- **Accesso alla memoria e alle risorse del processo:** condiviso con gli altri thread appartenenti allo stesso processo.

Benefici dell'uso dei Thread

- Creazione e terminazione più rapide rispetto ai processi.
- Cambiare esecuzione tra thread dello stesso processo è più veloce che tra processi diversi.

- Comunicazione più efficiente tra thread dello stesso processo.
- Permette elaborazioni asincrone, migliorando la velocità e la modularità del software.

Nota: La schedulazione è gestita a livello di thread, ma la sospensione e la terminazione dipendono dal processo.

Stati del Thread e Operazioni

Un thread può trovarsi nei seguenti stati:

- **In esecuzione**
- **Pronto** (in attesa di CPU)
- **In attesa** (bloccato per I/O o sincronizzazione).

Le operazioni principali sui thread sono:

- **Creazione**
- **Sospensione** (block)
- **Riattivazione** (unblock)
- **Terminazione.**

Sincronizzazione dei Thread

I thread di un processo condividono lo spazio di indirizzamento e le risorse. Questo può creare problemi di **concorrenza**, come la corruzione dei dati quando due thread accedono contemporaneamente alla stessa risorsa.

Per evitare problemi di accesso simultaneo è necessaria la **sincronizzazione** dei thread.

Thread a Livello Utente vs. Thread a Livello Kernel

Thread a livello utente

La gestione avviene a livello di applicazione.

Il kernel non è a conoscenza dell'esistenza dei thread.

Vantaggi:

- Più veloce, perché non richiede passaggi a modalità kernel.
- Maggiore portabilità.

Svantaggi:

- Se un thread effettua una chiamata di sistema bloccante, blocca l'intero processo.
- Non si possono sfruttare processori multipli.

Soluzioni:

- **Jacketing:** trasforma una chiamata di sistema bloccante in una non bloccante.
- Programmare l'applicazione con più processi indipendenti.

Thread a livello kernel

Il kernel gestisce direttamente i PCB dei processi e dei thread.

La schedulazione avviene sulla base dei thread.

Vantaggi:

- I thread possono essere eseguiti su diversi processori.
- Il blocco di un thread non blocca l'intero processo.

Svantaggi:

- I passaggi tra modalità utente e kernel generano overhead.

Modelli di Gestione dei Thread

Esistono diversi modelli per la gestione dei thread:

1. **1:1** - Ogni thread di esecuzione è un processo indipendente con il proprio spazio di indirizzamento (es. UNIX tradizionale).
2. **M:1** - Un processo può avere più thread, ma il kernel vede solo un singolo thread attivo (es. Windows NT, Solaris, Linux).
3. **1:M** - Un thread può migrare tra più processi, utile per sistemi distribuiti.
4. **M:N** - Unisce gli approcci precedenti, consentendo esecuzione parallela e flessibilità nella gestione delle risorse (es. TRIX).

Multicore e Multithreading

Con l'aumento dei processori multicore, il multithreading assume un ruolo sempre più importante.

La

legge di Amdahl descrive il limite del miglioramento delle prestazioni in base alla frazione di codice che può essere parallelizzata:

$$\text{Speedup} = \frac{\text{tempo di esecuzione su un processore}}{\text{tempo di esecuzione su } N \text{ processori}} = \frac{1}{(1 - f) + \frac{f}{N}}$$

- f è la frazione di codice parallelizzabile.
- N è il numero di processori.

Se una grande parte del codice è sequenziale, l'incremento di velocità è limitato.

Chi beneficia del multithreading?

- **Applicazioni multithread native:** poche istanze di processi con molti thread.
- **Applicazioni multiprocesso:** molteplici processi indipendenti.
- **Applicazioni Java:** la JVM è multithread, permettendo una gestione ottimizzata della concorrenza.
- **Multiple istanze di un'applicazione:** ogni istanza può eseguire operazioni parallele, sfruttando i thread.

Threads in UNIX

In UNIX, il kernel non distingue tra processi e thread. Ogni unità di esecuzione (processo o thread) è gestita tramite un **Task Control Block (TCB)**:

- `fork()` : crea un nuovo task duplicando il TCB.
- `clone()` : crea un nuovo task con riferimenti al TCB del processo padre (più efficiente).

Mutua Esclusione

Gestione dei Processi e Concorrenza

La concorrenza si manifesta in diversi contesti:

- **Multiprogrammazione:** più processi in esecuzione su un singolo processore.
- **Multiprocessing:** più processi eseguiti su un sistema con più processori.
- **Processi distribuiti:** processi eseguiti su più processori distribuiti (*cluster*).

La concorrenza è fondamentale in tre aree:

1. **Applicazioni multiple:** più programmi in esecuzione simultanea.
2. **Applicazioni strutturate:** suddivisione di un programma in più processi/thread cooperanti.
3. **Struttura del sistema operativo:** i componenti del sistema operativo funzionano in modo concorrente.

Principi della Concorrenza

L'elaborazione concorrente introduce complessità:

- Su **uniprocessore**, i processi si alternano.
- Su **multiprocessore**, i processi possono essere eseguiti in parallelo.

La velocità di esecuzione dei processi è imprevedibile e dipende da:

- Attività di altri processi.
- Gestione delle interruzioni.
- Politiche di schedulazione.

Difficoltà della concorrenza

- **Condivisione di risorse globali.**
- **Allocazione efficiente delle risorse.**
- **Debugging complesso.**

Race Condition

Si verifica una **race condition** quando più processi o thread accedono contemporaneamente a dati condivisi e l'ordine di esecuzione influisce sul risultato finale.

Esempio

- Due processi P1 e P2 assegnano rispettivamente 1 e 2 alla stessa variabile **a**.
- Il valore finale di **a** dipende da quale processo termina per ultimo.
- Se **P1** esegue prima **b = b + c** e poi **P2** esegue **c = b + c**, il risultato sarà diverso rispetto alla sequenza inversa.

Problemi della Concorrenza

Il sistema operativo deve gestire:

- **Tenere traccia dei processi** (PCB - Process Control Block).
- **Allocare e deallocare risorse:** tempo di CPU, memoria, file, dispositivi I/O.
- **Proteggere i dati e le risorse:** evitare interferenze involontarie tra processi.
- **Garantire risultati corretti** indipendentemente dalla velocità di esecuzione relativa dei processi concorrenti.

Competizione per le Risorse

I processi concorrenti possono entrare in **conflitto** per l'uso delle risorse, causando problemi come:

1. **Mutua esclusione:** quando più processi o thread condividono una risorsa, la necessità di accedere simultaneamente a questa risorsa può causare conflitti e danneggiamenti.
2. **Stallo (deadlock):** due o più processi rimangono bloccati aspettando risorse detenute dagli altri.
3. **Starvation:** un processo può non ricevere mai le risorse necessarie a causa di una gestione sbilanciata.

```
void P1() {
    while (true) {
        entercritical(Ra);
        /* Sezione critica */
        exitcritical(Ra);
    }
}
```

```
void P2() {
    while (true) {
        entercritical(Ra);
        /* Sezione critica */
        exitcritical(Ra);
    }
}
```

Cooperazione tra Processi

I processi possono collaborare tramite:

1. **Condivisione di memoria:** accesso a variabili condivise (richiede sincronizzazione).
2. **Scambio di messaggi:** i processi inviano e ricevono dati tramite il sistema operativo.

La comunicazione tra processi aiuta a sincronizzare e coordinare le attività, evitando la necessità di mutua esclusione, ma può causare **stallo o starvation**.

Requisiti per la Mutua Esclusione

Un meccanismo di mutua esclusione efficace deve garantire:

1. Un solo processo alla volta nella **sezione critica**.
2. **Nessuna interferenza** tra processi al di fuori della sezione critica.
3. Assenza di **deadlock** e **starvation**.
4. Accesso **equo** alla sezione critica.
5. **Indipendenza** dalla velocità relativa dei processi.
6. **Permanenza limitata** nella sezione critica.

Mutua Esclusione con Supporto Hardware

1. Disabilitazione delle Interruzioni

Su uniprocessore, assicura che la sezione critica sia portata a termine senza interruzioni.

- **Problemi:**

- Rallenta il sistema.
- Non applicabile su multiprocessore.

2. Istruzioni Atomiche

Alcuni processori offrono operazioni **atomiche** che garantiscono la sicurezza dell'accesso alle risorse condivise.

- **Vantaggi:**

- Funziona su qualsiasi numero di processi.
- Applicabile su sistemi uniprocessore e multiprocessore.

- **Svantaggi:**

- Introduce **busy-waiting** (attesa attiva).
 - Può causare **deadlock** e **starvation**.
-

Mutua Esclusione - 2

Algoritmo di Dekker

L'algoritmo di **Dekker** è uno dei primi metodi per ottenere la mutua esclusione tra due processi senza l'uso di supporto hardware speciale. Deve garantire:

- **Mutua esclusione**: solo un processo alla volta accede alla sezione critica.
- **Assenza di stallo**: nessun processo rimane bloccato indefinitamente.
- **Assenza di starvation**: ogni processo ha la possibilità di accedere alla sezione critica.

Problemi

Primo tentativo: uso di una variabile di turno

```
while (turn != 0) {}  
/* sezione critica */  
turn = 1;
```

Il processo 1 può rimanere bloccato se il processore 0 non gli cede mai il turno

Secondo tentativo: flag individuali

```
while (flag[1]) {}  
flag[0] = true;  
/* sezione critica */  
flag[0] = false;
```

Possibile deadlock se entrambi i processi impostano il proprio flag su **true** contemporaneamente.

Terzo tentativo: flag prima del controllo

```
flag[0] = true;  
while (flag[1]) {}  
/* sezione critica */  
flag[0] = false;
```

Ancora possibilità di deadlock se entrambi i processi impostano **flag** contemporaneamente.

Soluzione finale

```
boolean flag [2];  
int turn;  
void P0()  
{  
    while (true) {  
        flag [0] = true;  
        while (flag [1]) {  
            if (turn == 1) {  
                flag [0] = false;  
                while (turn == 1) {}  
                flag [0] = true;  
            }  
        }  
        /* critical section */;  
        turn = 1;  
        flag [0] = false;  
        /* remainder */;  
    }  
}
```

Introduzione di **turn** per regolare l'accesso e risolvere i deadlock.

Algoritmo di Peterson

Versione migliore di Dekker.

```
boolean flag[2];
int turn;

void P0() {
    while (true) {
        flag[0] = true;
        turn = 1;
        while (flag[1] && turn == 1);
        /* sezione critica */
        flag[0] = false;
    }
}
```

Semafori

Un **semaforo** è una variabile intera positiva che regola l'accesso alle risorse condivise, controllata attraverso:

- **semWait** decrementa il valore del semaforo e blocca il processo se il valore diventa negativo.
- **semSignal** incrementa il valore del semaforo e sblocca un processo in attesa.

```
struct semaphore {
    int count;
    queueType queue;
};

void semWait(semaphore s) {
    s.count--;
    if (s.count < 0) {
        /* Blocca il processo e lo inserisce nella coda */
    }
}

void semSignal(semaphore s) {
    s.count++;
    if (s.count <= 0) {
        /* Rimuove un processo dalla coda e lo sblocca */
    }
}
```

Semaforo binario

Variabile con valore binario (0 o 1):

- **semWaitB** se è 1 lo porta a 0, altrimenti è sospeso.

- `semSignalB` verifica l'attesa dei processi nella **coda** (*FIFO*), se affermativo uno dei processi viene riattivato, altrimenti il valore del semaforo diventa 1.

```
struct binary_semaphore {
    enum {zero, one} value;
    queueType queue;
};
```

Messaggi

Quando i processi interagiscono c'è bisogno di **sincronizzazione** (per realizzare mutua esclusione) e **comunicazione**.

Questo avviene tramite uno **scambio di messaggi** attraverso due primitive:

1. `send (destination, message)` → un processo invia messaggi ad un **destinatario**.
2. `receive (source, message)` → un processo riceve informazioni indicando il **mittente** e l'indirizzo in cui salvare il messaggio.

Sincronizzazione

Quando il processo esegue la `receive` se è presente il messaggio continua l'esecuzione, altrimenti o si sospende o continua senza riceverlo.

- **Send bloccante e Receive bloccante:** Mittente e destinatario bloccati fino al completamento dell'operazione.
- **Send non bloccante e Receive bloccante:** il mittente procede ma il destinatario è bloccato fino alla ricezione del messaggio.
- **Send non bloccante e Receive non bloccante:** nessuno dei due è bloccato.

Indirizzamento

1. **Indirizzamento diretto** → la Send include il PID del destinatario, la Receive può avere:
 - a. Indirizzamento esplicito con destinatario designato esplicitamente.
 - b. Indirizzamento implicito con parametro `sender` che restituisce il PID del mittente.
2. **Indirizzamento indiretto** → presenza di una **mailbox** condivisa contenente tutti i messaggi in cui il mittente invia un messaggio ed il destinatario lo preleva.

Formato del messaggio

- **Header:**
 - Message type
 - Destination ID
 - Source ID
 - Message Length
 - Control information
- **Body** con il messaggio.

Gestione delle code

Gestione di tipo FIFO con priorità per **tipo di messaggio** o **designata dal mittente**.

Il destinatario può ispezionare la coda e scegliere il messaggio.

Problema del Produttore-Consumatore

In un **buffer condiviso**, il produttore inserisce dati mentre il consumatore li preleva. Il sistema deve:

- Evitare che il produttore inserisca dati in un buffer pieno.
- Evitare che il consumatore prelevi dati da un buffer vuoto.

```
semaphore n = 0, s = 1;
```

```
void producer() {  
    while (true) {  
        produce();  
        semWait(s);  
        append();  
        semSignal(s);  
        semSignal(n);  
    }  
}
```

```
void consumer() {  
    while (true) {  
        semWait(n);  
        semWait(s);  
        take();  
        semSignal(s);  
        consume();  
    }  
}
```

Monitor

Modulo software simile ai semafori disponibile in vari linguaggi di programmazione e formato da:

1. **Dati locali** → accessibili solo attraverso le procedure del monitor.
2. Sequenza di **inizializzazione**.
3. Una o più **procedure**.

I processi accedono invocando una delle procedure ed uno soltanto può essere in esecuzione nel monitor.

Sincronizzazione

Le **variabili di condizione** gestiscono la sincronizzazione e sono accessibili solo all'interno del monitor attraverso due funzioni:

1. **cwait(c)** → sospende l'esecuzione del processo che l'ha eseguita sulla condizione **c**.
2. **csignal(c)** → riattiva uno dei processi in attesa sulla condizione **c** (se presenti).

Problema dei lettori e scrittori

Alcuni processi leggono (lettori) e gli altri scrivono.

I lettori possono leggere i dati simultaneamente, un solo scrittore scrive e, durante la scrittura, nessuno può leggere.

Stallo

Introduzione

Un insieme di processi è in **stallo (deadlock)** se ognuno è in attesa di un evento che può essere generato solo da un altro processo dell'insieme.

Esempio pratico: Immaginiamo due processi, **P1** e **P2**. **P1** ha il controllo della risorsa A e sta aspettando la risorsa B, mentre **P2** ha il controllo della risorsa B e sta aspettando la risorsa A. Nessuno dei due può procedere, causando uno stallo.

Categorie di risorse

Risorse riutilizzabili

Utilizzabili **da un solo processo** alla volta e non vengono distrutte dopo l'utilizzo (processore, canali I/O, memoria...).

Risorse consumabili

Create e distrutte (consumate).
(interruzioni, segnali, buffer...)

Condizioni per lo Stallo

Perché si verifichi uno stallo, devono essere presenti contemporaneamente quattro condizioni:

1. **Mutua esclusione:** Una risorsa può essere usata da un solo processo alla volta.
2. **Possesso e attesa:** Un processo in possesso di una risorsa sta aspettando di ottenere un'altra risorsa già allocata ad un altro processo.
3. **Assenza di prerilascio:** Le risorse non possono essere sottratte forzatamente ai processi; devono essere rilasciate volontariamente.
4. **Attesa circolare (Condizione determinante):** Esiste una catena circolare di processi, ognuno dei quali attende una risorsa posseduta dal successivo nella catena.

Prevenire lo stallo

Metodi indiretti

Prevengono il presentarsi di una delle tre condizioni necessarie.

- **Mutua esclusione** → impossibile disabilitarla per risorse seriali, solo in rari casi.
- **Possesso e attesa** → imporre ai processi un'unica richiesta globale di tutte le risorse, sospendendolo finché non le ha ottenute tutte simultaneamente (**assegnazione statica**). Tuttavia genera processi sospesi e risorse assegnate non utilizzate.
- **Assenza di prerilascio** → imporre ai processi in attesa di una risorsa di rilasciare quelle che possiede.

Metodi diretti

Prevengono il presentarsi dell'attesa circolare.

- **Assegnazione ordinata** → si definisce un ordine totale sulle categorie di risorse: un processo cui siano già state assegnate risorse può richiedere solo quelle di categorie successive. Tuttavia genera risorse assegnate e non utilizzate e processi sospesi in attesa.

Evitare lo stallo

- **Deadlock prevention:** limitazioni sulle richieste di risorse.
- **Deadlock avoidance:** limitazioni sulle assegnazioni di risorse.

Rifiuto di esecuzione per i processi le cui richieste potrebbero determinare stallo.

Rifiuto di allocazione per le richieste di risorse che potrebbero determinare stallo.

Rifiuto di esecuzione

Inizia un nuovo processo P_{n+1} se e solo se

$$\forall j, R_j \geq C_{n+1,j} + \sum_{i=1}^n C_{ij}$$

cioè ci sono risorse sufficienti per soddisfare tutti i processi.

Tuttavia assume che tutte le richieste di tutti i processi debbano essere soddisfatte simultaneamente.

Algoritmo del Banchiere

1. Ogni processo dichiara il numero massimo di risorse di cui potrebbe aver bisogno.
2. Il sistema verifica se, concedendo risorse, rimane in uno stato sicuro.
3. Se sì, le risorse vengono allocate; altrimenti, il processo deve attendere.

```

W ← V
while (c'è un processo P[i] non spuntato t.c. C[i][j] - A[i][j] ≤ W[j] per ogni j)
    spunta P[j]
    for all j
        W[j] ← W[j] + A[i][j]
if (tutti i processi sono spuntati)
    return SICURO
else
    return INSICURO

```

Tramite questo algoritmo **non richiede prerilascio** e ripristino del processo ed è meno restrittivo della prevenzione.

Tuttavia vanno stabilite in anticipo le risorse e sono allocabili in numero fisso, inoltre i processi devono essere indipendenti e non possono terminare prima di aver rilasciato le risorse.

Rilevare lo stallo

Con la **deadlock detection** le risorse sono assegnate senza restrizioni e ci sono periodiche verifiche dell'assenza di attesa circolare, con algoritmi semplici e ripristini precoci.

Tuttavia verifiche frequenti consumano tempo di elaborazione.

Algoritmo di rilevazione dello stallo

Matrice di allocazione A

Vettore risorse disponibili V

Matrice delle richieste Q.

```
W ← V
spunta tutti i processi Pi tali che Aij = 0 per ogni j
while c'è un processo Pi non spuntato, tale che Qij ≤ Wj per ogni j
  spunta Pi
  for all j
    Wj ← Wj + Aij
```

Gli eventuali processi non spuntati sono tutti in stallo.

Strategie di ripristino

- Terminare tutti i processi in stallo.
- Ripristinare tutti i processi in stallo ad uno stato precedente (**checkpoint**), rischiando però di riproporre lo stallo.
- Terminare un processo alla volta finché lo stallo non scompare (consumo minimo).
- Revocare una risorsa alla volta finché lo stallo non scompare (costo minimo).

Strategia integrata

- Raggruppare le risorse in classi.
- Usare l'assegnazione ordinata per prevenire attesa circolare fra le classi di risorse.
- In ogni classe utilizzare il metodo più appropriato.

Classi di risorse

1. **Spazio di swap** assegnazione statica (o rifiuti di esecuzione).
2. **Risorse dei processi** (dispositivi) algoritmo del banchiere (o assegnazione ordinata).
3. **Memoria principale** prevenzione tramite prerilascio.
4. **Risorse interne** (canali I/O) assegnazione ordinata.

Gestione della Memoria

Gestione della memoria

La gestione della memoria prevede vari compiti:

1. Rilocazione
2. Protezione
3. Condivisione
4. Organizzazione logica
5. Organizzazione fisica.

Rilocazione

La rilocazione serve a trasferire programmi dalla memoria principale alla secondaria (**swapping**) in aree differenti.

Statica

Si usa un **loader** rilocante per caricare un processo nella medesima partizione: al caricamento tutti i riferimenti a indirizzi relativi saranno sostituiti da indirizzi assoluti.

Dinamica

Un processo può occupare partizioni diverse nel corso dell'esecuzione.

Viene caricata l'immagine del processo in una partizione, trasferita in memoria secondaria e poi ricaricata in un'altra partizione.

Le locazioni variano quindi ogni volta che il processo subisce uno swap.

Protezione

Non è prevedibile conoscere la locazione del programma in memoria, dev'essere verificata **al tempo di esecuzione**.

I meccanismi che supportano la rilocazione supportano anche la protezione.

Condivisione

Il gestore della memoria permette un accesso controllato alle **aree di memoria condivisa**, così da permettere ai processi di un programma di accedere allo stesso codice, senza compromettere la protezione.

I meccanismi che supportano la rilocazione supportano anche la condivisione.

Organizzazione logica

I programmi sono scritti linearmente in moduli, scritti e compilati indipendentemente.

I moduli possono essere sola lettura o solo scrittura e condivisibili tramite la **segmentazione**.

Organizzazione fisica

Memoria principale

- veloce
- volatile
- costosa

Memoria secondaria

- permanente
- lenta
- economica

L'**overlaying** è il processo di trasferimento di un blocco di codice di programma o altri dati nella memoria principale, sostituendo ciò che è già memorizzato.

Tramite l'overlaying il programma assegna ai vari moduli la stessa regione di memoria.

Partizionamento della memoria

Il gestore della memoria carica i processi per eseguirli, realizza la memoria virtuale e si basa su **partizionamento**, **segmentazione** e **paginazione**.

Partizionamento

Ormai è obsoleto e non prevede memoria virtuale, bensì partizioni **fisse**, cioè di dimensioni uguali, o **variabili**.

Partizioni fisse

Gli svantaggi sono:

- necessità di overlay per un programma troppo lungo per una partizione;
- limite al numero di processi attivi;
- un programma occupa un'intera partizione;
- **frammentazione interna**: spreco di spazio se il blocco dati è più piccolo di una partizione.

Partizioni dinamiche

Partizioni di lunghezza e numero variabili, create dinamicamente quando un processo viene caricato in memoria.

Svantaggio principale è la **frammentazione esterna**: si verifica quando la quantità totale di memoria allocabile è sufficiente per esaudire una richiesta, ma la memoria disponibile non è contigua e quindi non può essere allocata.

Algoritmi di allocazione

1. **Best-fit**: sceglie il blocco di **dimensione più prossima** alla dimensione richiesta.
2. **First-fit**: sceglie il **primo blocco di dimensione sufficiente** a partire *dall'inizio della memoria*.
3. **Next-fit**: sceglie il primo blocco di dimensione sufficiente a partire **dall'ultima partizione creata**.

Buddy system

Compromesso tra partizioni fisse e dinamiche con blocchi di dimensione 2^K , $L \leq K \leq U$ ove 2^U è lo spazio allocabile.

2^L è la dimensione minima dei blocchi allocabili.

Un programma di dimensione ℓ , $2^{K-1} < \ell \leq 2^K$ viene allocato in un blocco di dimensione 2^K .

Se non ce ne sono un blocco di dimensione 2^{K+1} viene spezzato in due **buddies** di dimensione 2^K . Non appena si liberano vengono riuniti in un blocco di dimensione doppia.

Indirizzi

Indirizzo logico: riferimento ad una locazione di memoria indipendente dall'assegnazione della memoria principale al processo.

Indirizzo fisico: effettiva posizione del dato nella memoria principale.

Indirizzo relativo: indirizzo logico espresso come posizione relativa ad un altro indirizzo noto.

Paginazione

La memoria principale viene suddivisa in blocchi di dimensione fissa detti **frame**.

I programmi sono divisi in blocchi della stessa dimensione (**pagine**), caricate in frame non necessariamente contigui.

Tabella delle pagine

Il S.O. gestisce una tabella per processo, contenente la **posizione del frame per ogni pagina** del processo.

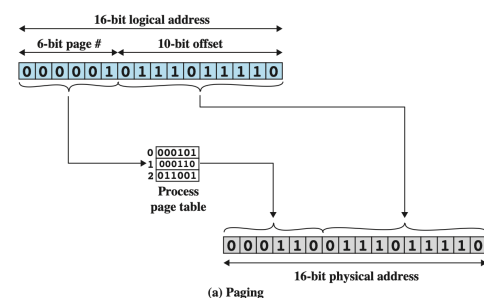
Il processore accede alla tabella per produrre l'indirizzo fisico.

Un indirizzo logico generato dalla CPU è generalmente suddiviso in due parti:

1. **Numero di pagina (p):** utilizzato come indice nella tabella delle pagine per individuare il frame fisico corrispondente.
2. **Offset all'interno della pagina (d):** specifica la posizione esatta all'interno della pagina.

La traduzione da indirizzo logico a indirizzo fisico avviene nel seguente modo:

1. La CPU genera un indirizzo logico composto da p e d.
2. Il numero di pagina p viene utilizzato per accedere alla tabella delle pagine e ottenere il numero di frame fisico corrispondente.
3. L'offset d viene combinato con l'indirizzo base del frame fisico per ottenere l'indirizzo fisico finale.



Segmentazione

Un programma viene diviso in **segmenti** di lunghezza differente e limitata, ciascuno rappresentante un'unità logica del programma, come il codice, i dati, lo stack, ecc..

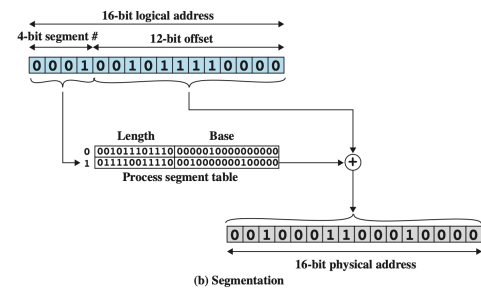
L'indirizzo logico è diviso in:

1. **Indice di segmento:** identifica il segmento specifico all'interno dello spazio di indirizzamento del processo.
2. **Offset:** specifica la posizione esatta all'interno del segmento.

Il programmatore deve tener conto della lunghezza massima dei segmenti.

La traduzione degli indirizzi avviene nel seguente modo:

1. gli n bits più a sinistra danno l'indice di segmento;
2. utilizza l'indice di segmento per trovare l'indirizzo fisico a cui inizia il segmento (**indirizzo base**);
3. compara l'offset nei rimanenti m bits con la lunghezza del segmento (**indirizzo limite**).
Se è più grande allora non è valido e si incorre nell'**eccezione di indirizzo**;
4. l'indirizzo fisico è la somma dell'indirizzo base e l'offset.



Memoria Virtuale

Hardware e strutture di controllo

Per la gestione dinamica della memoria abbiamo bisogno di:

- Rilocalizzazione dinamica run-time.
- Spezzare il processo in parti che non necessitano di essere caricate in spazi contigui della memoria (non per forza memoria principale).

Esecuzione

1. Il SO carica in memoria **solo poche pagine o segmenti** del programma (**Resident set**: quantità di memoria effettivamente mappata).
2. Interruzione se l'indirizzo richiesto non è in memoria.
3. Il SO sospende il processo ed il processore viene assegnato ad un altro.
4. La pagina o segmento contenente l'indirizzo logico richiesto viene caricata in memoria (**DMA: Direct Memory Access**).
5. Interruzione al termine del caricamento dopo della quale il SO riattiva il processo.

Vantaggi

- Più spazio per i processi ed aumento dei processi pronti ⇒ incremento della multiprogrammazione.
- Non è necessario l'overlay perché un processo può essere più lungo dell'intera memoria principale.
- Il SO carica solo le parti necessarie di volta in volta.
- Il programmatore non deve conoscere la quantità di memoria disponibile.

Memoria reale: memoria principale (RAM).

Memoria virtuale: realizzata dal SO con un caricamento dinamico.

Paginazione Semplice	Paginazione con Memoria Virtuale	Segmentazione Semplice	Segmentazione con Memoria Virtuale
La memoria principale è suddivisa in piccoli blocchi di dimensione fissa chiamati frame	La memoria principale è suddivisa in piccoli blocchi di dimensione fissa chiamati frame	La memoria principale non è partizionata	La memoria principale non è partizionata
Il programma è suddiviso in pagine dal compilatore o dal sistema di gestione della memoria	Il programma è suddiviso in pagine dal compilatore o dal sistema di gestione della memoria	I segmenti del programma sono specificati dal programmatore al compilatore (cioè, la decisione è presa dal programmatore)	I segmenti del programma sono specificati dal programmatore al compilatore (cioè, la decisione è presa dal programmatore)
Frammentazione interna all'interno dei frame	Frammentazione interna all'interno dei frame	Frammentazione esterna	Frammentazione esterna

Paginazione Semplice	Paginazione con Memoria Virtuale	Segmentazione Semplice	Segmentazione con Memoria Virtuale
Il sistema operativo deve mantenere una tabella delle pagine per ogni processo, che indichi quale frame contiene ciascuna pagina	Il sistema operativo deve mantenere una tabella delle pagine per ogni processo, che indichi quale frame contiene ciascuna pagina	Il sistema operativo deve mantenere una tabella dei segmenti per ogni processo, che indichi l'indirizzo di caricamento e la lunghezza di ciascun segmento	Il sistema operativo deve mantenere una tabella dei segmenti per ogni processo, che indichi l'indirizzo di caricamento e la lunghezza di ciascun segmento
Il sistema operativo deve mantenere una lista dei frame liberi	Il sistema operativo deve mantenere una lista dei frame liberi	Il sistema operativo deve mantenere una lista dei buchi liberi nella memoria principale	Il sistema operativo deve mantenere una lista dei buchi liberi nella memoria principale
Il processore utilizza il numero di pagina e l'offset per calcolare l'indirizzo assoluto	Il processore utilizza il numero di pagina e l'offset per calcolare l'indirizzo assoluto	Il processore utilizza il numero di segmento e l'offset per calcolare l'indirizzo assoluto	Il processore utilizza il numero di segmento e l'offset per calcolare l'indirizzo assoluto
Tutte le pagine di un processo devono essere in memoria principale per poter essere eseguito, a meno che non si utilizzino tecniche di overlay	Non tutte le pagine di un processo devono essere in memoria principale per poter essere eseguito. Le pagine possono essere caricate all'occorrenza	Tutti i segmenti di un processo devono essere in memoria principale per poter essere eseguito, a meno che non si utilizzino tecniche di overlay	Non tutti i segmenti di un processo devono essere in memoria principale per poter essere eseguito. I segmenti possono essere caricati all'occorrenza
	Il caricamento di una pagina in memoria principale può richiedere la scrittura su disco di un'altra pagina		Il caricamento di un segmento in memoria principale può richiedere la scrittura su disco di uno o più segmenti

Principio di località

Trashing

Il trashing si verifica quando un sistema spende più tempo per la carica e scaricare dalla memoria, rispetto al tempo destinato all'esecuzione.

Per evitare il trashing si fanno ipotesi su quali parti di pagine o segmenti saranno necessarie in futuro.

Paginazione

L'hardware deve supportare la paginazione e la segmentazione.

Il SO deve includere il software per gestire il movimento delle pagine o dei segmenti fra memoria secondaria e memoria principale.

Il programma genera un **indirizzo virtuale**, che è suddiviso in due parti:

- **Page number (numero di pagina):** identifica la pagina logica all'interno del processo.
- **Offset:** indica la posizione esatta all'interno di quella pagina.

L'indirizzo ha n bit.

Ogni processo ha una **tabella delle pagine**, ogni record è fatto così:

- **P:** present bit
- **M:** modified bit

- **Bit di controllo**
- **Frame number.**

Rilocazione (support HW)

Questa è la parte centrale che si occupa della traduzione da indirizzo virtuale a indirizzo fisico.

Registro del puntatore alla tabella delle pagine

Il sistema ha un **registro speciale** chiamato **Page Table Ptr** (puntatore alla tabella delle pagine), che contiene l'indirizzo base della **tabella delle pagine** del processo in esecuzione.

Tabella delle pagine

Ogni processo ha la sua tabella, che mappa ogni **numero di pagina logico** al **numero di frame fisico** corrispondente.

In pratica, si usa il **Page #** per accedere alla posizione nella tabella dove è indicato il **Frame #** (numero di frame in memoria principale dove si trova fisicamente quella pagina).

Traduzione dell'indirizzo

Una volta ottenuto il numero del frame fisico, lo si **combina con lo stesso offset** dell'indirizzo virtuale, generando così l'**indirizzo fisico**.

L'indirizzo fisico ha **m bit** (in generale $m > n$).

Memoria Principale

L'indirizzo fisico punta ora a una posizione specifica in memoria:

- Il **Frame #** individua il blocco (frame) fisico.
- L'**Offset** dice dove, all'interno di quel frame, si trova esattamente il dato richiesto.

Tabella delle pagine a due livelli

La tabella delle pagine a due livelli è composta da:

1. Tabella Esterna (Page Directory):

- Contiene puntatori alle tabelle interne.
- Ogni voce corrisponde a una sezione dello spazio di indirizzamento virtuale.

2. Tabelle Interne (Page Tables):

- Ogni tabella interna gestisce un sottoinsieme delle pagine virtuali.
- Contiene le mappature tra pagine virtuali e frame fisici.

L'indirizzo virtuale viene suddiviso in tre parti:

- **p1 (10 bit):** indice nella tabella delle pagine di primo livello.
- **p2 (10 bit):** indice nella tabella delle pagine di secondo livello.
- **d (12 bit):** offset all'interno della pagina.

1. Generazione dell'Indirizzo Virtuale:

Il processore genera un indirizzo virtuale composto da p1, p2 e d.

2. Accesso alla Tabella di Primo Livello:

Utilizzando p1, la MMU accede alla **tabella delle pagine di primo livello** per ottenere l'indirizzo base della corrispondente **tabella di secondo livello**.

3. Accesso alla Tabella di Secondo Livello:

Con p2, la MMU accede alla **tabella delle pagine di secondo livello** per ottenere il **numero di frame fisico** associato alla pagina virtuale.

4. Calcolo dell'Indirizzo Fisico:

Il numero di frame fisico ottenuto viene combinato con l'offset d per formare l'**indirizzo fisico** finale.

5. Accesso alla Memoria Fisica:

L'indirizzo fisico viene utilizzato per accedere alla memoria principale e recuperare o scrivere i dati desiderati.

Tabella delle pagine invertita

Il numero di pagina è mappato in una tabella hash: il valore di hash punta alla tabella delle pagine **invertita** ($\Rightarrow$ *le entries sono in ordine di frame anziché di pagina*), contente solo le pagine caricate in memoria principale.

I campi sono:

- **Numero di pagina.**
- **PID** del processo.
- **Bit di controllo.**
- **Puntatore** all'entry successiva.

Quando un processo accede a un indirizzo virtuale:

1. La **Memory Management Unit (MMU)** riceve l'indirizzo virtuale e il PID del processo.
 2. La MMU cerca nella tabella delle pagine invertita una entry che corrisponda alla coppia (**PID, numero di pagina virtuale**).
 3. Se trova una corrispondenza, utilizza l'indice della entry (che corrisponde al numero del frame fisico) per calcolare l'indirizzo fisico finale, aggiungendo l'offset.
 4. Se non trova una corrispondenza, si verifica un **page fault**, e il sistema operativo deve gestire il caricamento della pagina richiesta.
-

Translation Lookaside Buffer (TLB)

Poiché la ricerca nella tabella delle pagine invertita può essere lenta (potenzialmente richiede una scansione dell'intera tabella), si utilizza una **cache associativa** chiamata **Translation Lookaside Buffer (TLB)**.

Il TLB memorizza le traduzioni recenti da indirizzi virtuali a indirizzi fisici, permettendo una traduzione rapida senza accedere alla tabella principale.

1. Generazione dell'Indirizzo Virtuale:

Il processore genera un indirizzo virtuale composto da un numero di pagina virtuale e un offset.

2. Consultazione del TLB:

- La MMU verifica se il numero di pagina virtuale è presente nel TLB.

- Se presente (**TLB hit**), la corrispondente cornice di pagina fisica viene immediatamente utilizzata.
- Se non presente (**TLB miss**), la MMU accede alla tabella delle pagine per ottenere la mappatura necessaria.

3. Aggiornamento del TLB:

Dopo un TLB miss, la nuova mappatura viene inserita nel TLB per velocizzare futuri accessi.

4. Calcolo dell'Indirizzo Fisico:

La cornice di pagina fisica ottenuta viene combinata con l'offset per determinare l'indirizzo fisico finale.

Dimensione delle pagine

La frammentazione interna è ridotta da pagine di piccole dimensioni, esse aumentano la dimensione della tabella delle pagine.

La dimensione delle pagine influisce sulla frequenza degli errori di pagina.

Il TLB è meno efficiente con pagine di piccole dimensioni.

Segmentazione

Memoria composta da multipli spazi di indirizzamento (**segmenti**).

Vantaggi:

1. Gestione di strutture dati di dimensione variabile semplificata.
2. Ricompilazione di segmenti di programma.
3. Condivisione i dati fra processi.
4. Protezione.

Tabella dei segmenti

Ogni record è così composto:

- **Indirizzo** del segmento in memoria.
- **Lunghezza** del segmento.
- Flag **caricato**
- Flag **modificato**

Rilocazione

1. Validazione del Segmento:

La MMU verifica se il numero di segmento **sg** è valido confrontandolo con il registro **STLR (Segment Table Limit Register)**. Se **sg** è maggiore o uguale a **STLR**, viene generata un'eccezione.

2. Accesso alla Tabella dei Segmenti:

La MMU calcola l'indirizzo della voce corrispondente nella tabella dei segmenti sommando **sg** al registro **STBR (Segment Table Base Register)**.

3. Validazione dell'Offset:

La MMU confronta **off** con la lunghezza del segmento. Se **off** è maggiore o uguale alla lunghezza,

viene generata un'eccezione.

4. **Calcolo dell'Indirizzo Fisico:**

Se le validazioni hanno successo, la MMU somma l'offset off all'indirizzo base del segmento per ottenere l'indirizzo fisico finale.

Segmentazione paginata

Lo spazio logico è diviso in segmenti, ognuno diviso in pagina di dimensione fissa, uguale alla lunghezza dei frame della memoria principale.

1. **Accesso alla Tabella dei Segmenti:**

Utilizzando il numero di segmento s , si accede alla tabella dei segmenti per ottenere l'indirizzo base della tabella delle pagine corrispondente.

2. **Accesso alla Tabella delle Pagine:**

Con il numero di pagina p , si accede alla tabella delle pagine del segmento per ottenere il numero del frame fisico.

3. **Calcolo dell'Indirizzo Fisico:**

Il numero del frame fisico ottenuto viene combinato con l'offset d per determinare l'indirizzo fisico finale.

La segmentazione facilita l'implementazione della **protezione e della condivisione**:

- Ogni record della tabella dei segmenti contiene indirizzo base e lunghezza, per controllare accessi erranei.
 - La condivisione è implementata inserendo un riferimento allo stesso segmento nella tabella dei segmenti di più processi.
-

Memoria Virtuale - Politiche di Gestione

Software del gestore della memoria

La progettazione del gestore della memoria dipende da tre decisioni:

1. usare o meno tecniche di **memoria virtuale**;
2. **paginazione**, **segmentazione** o **segmentazione paginata**;
3. **algoritmi** utilizzati per la gestione della memoria.

Strategia di caricamento

Determina quando una pagina dev'essere caricata in memoria.

Paginazione a richiesta

Carica una pagina **solo quando c'è un riferimento ad un indirizzo della pagina**.

All'avvio del processo ci potrebbero essere molti errori di pagina, tuttavia il principio di località suggerisce che più pagine ci sono e meno sono gli errori.

Prepaginazione

Carica altre pagine oltre a quelle con l'errore di pagina, sfruttando le caratteristiche dei dispositivi di memoria secondaria: **pagine in aree contigue sono caricate in serie più velocemente**, con il rischio di prendere pagine non riferite.

Strategia di posizionamento

Determina la **posizione** della memoria principale **in cui la pagina o segmento verrà caricato**.

Per la segmentazione non paginata ci sono tre algoritmi (first fit, best fit e next fit).

Strategia di sostituzione

Seleziona la pagina da **sostituire** quando si deve caricare una nuova pagina, scegliendo quella che più **difficilmente sarà riferita in futuro**.

Più complessa è la politica, maggiore è l'overhead per implementarla.

Blocco del frame

Quando un frame è **locked (bloccato)**, la pagina caricata in quel frame non può essere scaricata:

1. il **kernel** del SO e le sue strutture di controllo sono caricate in frame bloccati,
2. **buffer I/O** e aree per interazioni real-time sono caricate in frame temporaneamente bloccati.

Il blocco si ottiene associando un *lock-bit* ad ogni frame nella tabella dei frame o delle pagine.

Algoritmo ottimo

Sostituisce la pagina che, in futuro, sarà riferita per ultima, ma è **irrealizzabile**.

Least Recently Used (LRU)

Sostituisce la pagina che non è stata riferita **per il tempo massimo**, dato che, per il principio di località, è quella con meno probabilità di essere riferita nel futuro.

L'idea è marcare ogni pagina col tempo dell'ultimo riferimento o organizzare le pagine in una coda FIFO, andando in contro ad un grande overhead.

First in First Out (FIFO)

Le pagine sono organizzate in un buffer circolare (coda FIFO) e sono rimosse in **ordine di caricamento**, sostituendo quella che è stata più tempo in memoria.

Strategia dell'orologio

Associa un **bit di utilizzo** ad ogni frame:

- 1 quando la pagina è caricata
- 1 ogni volta che è riferita

L'algoritmo scandisce circolarmente i frame:

- se il bit è 1 lo setta a 0
- se il bit è 0, il frame viene selezionato per lo scaricamento.

Buffer per pagine

Le pagine caricate sono inserite in due liste: lista delle **pagine libere** e lista delle **pagine modificate**.

Esse rimangono disponibili prima della sovrascrittura e le pagine modificate sono salvate a gruppi.

Resident set

Il SO decide **quante pagine** di ogni processo tenere in memoria. Meno frame sono allocati e più processi possono essere attivi. Pochi frame allocati incrementano gli errori di pagina.

Allocazione fissa

Il processo ha un **numero fisso di frame** in cui essere eseguito, ad ogni errore di pagina si scarica una pagina del processo.

Allocazione dinamica

Il numero di frame allocati a un processo **può variare**.

Ambito della strategia di sostituzione

Ambito locale

Sceglie la pagina da sostituire **tra le pagine residenti** del processo che ha determinato l'errore di pagina.

Ambito globale

Considera **tutte le pagine** non bloccate presenti in memoria.

Entrambi sono attivati da un errore di pagina.

Allocazione fissa, ambito locale

Stabilire lo spazio da assegnare al processo: **non troppo piccola** altrimenti avremo un alto numero di errori, **non troppo grande** per non avere pochi

Allocazione variabile, ambito globale

Il SO mantiene una **lista di frame liberi** ed in caso di errore di pagina un frame è aggiunto all'insieme residente. In assenza di frame liberi una qualunque pagina può essere scaricata.

processi in memoria e un rallentamento della CPU.

Allocazione variabile, ambito locale

Per ogni processo caricato vengono **allocati un numero di frame**, quando c'è un errore di pagina viene **sostituita una pagina** dall'insieme residente del processo che ha determinato l'errore.

Periodicamente si incrementa o decrementa lo spazio assegnato.

Working set

Il **working set** $W(t, \Delta)$ è l'insieme delle pagine riferite nell'intervallo di tempo virtuale $[t - \Delta, t]$.

La decisione di incrementare o diminuire la dimensione dell'insieme residente si basa sulla **valutazione delle future richieste** dei processi attivi e richiede dei criteri per la determinazione della dimensione dell'insieme e la tempistica delle variazioni.

Una strategia possibile è **controllare periodicamente** il working set di ogni processo, rimuovere dall'insieme tutte le pagine che non rientrano nel working set ed eseguire solo i processi che lo hanno in memoria.

Page fault frequency

Quando c'è un errore di pagina:

- se il tempo virtuale dall'errore precedente è inferiore ad una soglia, si aggiunge una pagina all'insieme residente;
- altrimenti si scaricano tutte le pagine con il bit di utilizzo a 0 e si azzera il bit delle rimanenti.

Variable-Interval Sampled Working Set (VSWS)

Valuta il working set ad intervalli di campionamento e ed è diretto da:

1. Durata minima dell'intervallo di campionamento
2. Durata massima dell'intervallo
3. Numero degli errori di pagina consentiti tra gli intervalli.

Politica di cleaning

Cleaning su richiesta

La pagina è salvata in memoria secondaria solo alla sua sostituzione.

Precleaning

Permette il salvataggio di pagine a lotti.

Politiche di caricamento

La **politica di caricamento** nella gestione della memoria è fondamentale per determinare quanti e quali processi possono risiedere simultaneamente nella memoria principale. Questa decisione influisce direttamente sul **grado di multiprogrammazione**, ovvero sul numero di processi attivi che il sistema può gestire contemporaneamente.

Un basso grado di multiprogrammazione, con pochi processi in memoria, può portare a situazioni in cui tutti i processi sono in attesa di risorse (ad esempio, I/O), lasciando la CPU inattiva e riducendo l'efficienza del sistema.

Al contrario, un eccessivo numero di processi in memoria può causare il **trashing**, una condizione in cui il sistema spende più tempo a gestire lo swapping delle pagine tra memoria principale e secondaria che a eseguire effettivamente i processi. Questo fenomeno degrada significativamente le prestazioni del sistema.

Per evitare il trashing, il sistema operativo può decidere di ridurre il grado di multiprogrammazione, scaricando uno o più processi dalla memoria. Le politiche per scegliere quali processi scaricare includono:

- **Processo con priorità minima:** viene rimosso il processo con la priorità più bassa.
 - **Processo che provoca errori di pagina:** si sceglie il processo che genera il maggior numero di page fault.
 - **Ultimo processo attivato:** viene scaricato l'ultimo processo caricato in memoria.
 - **Processo con l'insieme residente più piccolo:** si rimuove il processo che occupa meno spazio in memoria.
 - **Processo più grande:** viene scaricato il processo che occupa più memoria.
 - **Processo col massimo tempo di esecuzione rimanente:** si sceglie il processo che richiederà più tempo per completare l'esecuzione.
-

Scheduling

Lo **scheduling** è la componente del sistema operativo che si occupa di scegliere quale processo deve essere eseguito dalla CPU. Il suo obiettivo è quello di massimizzare le prestazioni del sistema ottimizzando parametri come il tempo di risposta, il throughput e l'efficienza del processore.

Tipi di scheduling

Esistono diversi tipi di scheduling, a seconda del momento e dello scopo:

- **Scheduling a lungo termine:** Decide quali processi ammettere nella lista dei processi da eseguire, controllando il grado di multiprogrammazione.
- **Scheduling a medio termine:** Decide quali processi caricare o scaricare dalla memoria principale, agendo parzialmente sul grado di multiprogrammazione.
- **Scheduling a breve termine:** Seleziona, tra i processi pronti, quello che deve essere eseguito immediatamente dalla CPU.
- **Scheduling I/O:** Seleziona tra i processi in attesa di I/O quale assegnare a un dispositivo disponibile.

Funzionamento e Transizioni degli Stati

Lo scheduling è strettamente connesso agli stati dei processi: `new`, `ready`, `running`, `blocked`, `exit`. Il sistema gestisce transizioni come il passaggio da `ready` a `running` (a cura dello scheduling a breve termine) o da `running` a `blocked`.

Schedulatore a lungo termine

Lo schedulatore a lungo termine determina **quali programmi possono entrare nel sistema**. Controlla il numero di processi in esecuzione, perché un numero eccessivo riduce la porzione di CPU assegnata a ciascuno, peggiorando le prestazioni.

Per selezionare i processi si può adottare diverse politiche come "first come, first served", priorità, tempo di esecuzione atteso o requisiti I/O.

Schedulatore a medio termine

Lo schedulatore a medio termine si occupa dello **swapping**: può sospendere e riprendere i processi, in modo da ottimizzare l'uso della memoria e controllare il grado di multiprogrammazione.

Il Dispatcher

Il **dispatcher** è il modulo che esegue materialmente la decisione dello scheduler a breve termine. Viene attivato da eventi come interruzioni di clock, completamenti I/O, chiamate di sistema o segnali (es. semafori).

Criteri di Scheduling a breve termine

Gli obiettivi dello scheduling variano a seconda del punto di vista:

- **Criteri orientati all'utente:** Riguardano la percezione degli utenti, ad esempio il tempo di risposta nei sistemi interattivi.
- **Criteri orientati al sistema:** Riguardano l'efficienza di utilizzo del processore, come la frequenza del completamento dei processi.

I criteri possono essere anche distinti tra *quantitativi* (misurabili, come il tempo di risposta) e *qualitativi* (meno misurabili, come la prevedibilità).

Parametri importanti per lo scheduling

Prestazioni

- **Tempo di risposta:** tempo che intercorre tra l'invio di una richiesta e l'inizio della risposta.
- **Tempo di turnaround:** tempo complessivo tra la richiesta e la fine dell'esecuzione.
- **Scadenze:** fondamentale nei sistemi real-time.

Utente

- **Prevedibilità:** importante per mantenere il comportamento del sistema stabile. Un dato job può essere eseguito nello stesso tempo e allo stesso costo, indipendentemente dal carico del sistema. Un'ampia variazione nel tempo di risposta o nel tempo di turnaround è fastidiosa per l'utente: potrebbe segnalare un'ampia oscillazione nel carico di lavoro del sistema oppure la necessità di adattare il sistema per evitare l'instabilità

Sistema

- **Throughput:** numero di processi completati per unità di tempo.
- **Utilizzazione del processore:** percentuale di tempo in cui la CPU è attivamente usata.
- **Fairness:** trattamento equo tra i processi.
- **Priorità:** dare precedenza ai processi più importanti.
- **Bilanciamento delle risorse:** evitare che alcune risorse rimangano inutilizzate.

Funzione di selezione

Determina quale processo selezionare per l'esecuzione, basandosi su **priorità**, esigenza di **risorse** e caratteristiche di **esecuzione**, tra cui:

- w = tempo di attesa
- e = tempo di esecuzione
- s = tempo stimati di esecuzione (compreso e).

Modo di decisione

- **Senza prerilascio:** l'esecuzione continua fino al termine o fino ad un'interruzione I/O.
- **Con prerilascio:** il processo può essere interrotto dal SO e posto in stato di pronto. Il prerilascio può avvenire quando arriva un nuovo processo, quando un'interruzione riattiva un processo sospeso o in base ad un clock.

Strategie di Scheduling

First Come First Served (FCFS)

È la politica più semplice: i processi sono serviti nell'**ordine di arrivo**, quando il processo in esecuzione termina o è sospeso, viene selezionato per l'esecuzione il processo che è stato più a lungo nella coda pronti. Penalizza i processi brevi e quelli I/O bound.

Round Robin

Il prerilascio è basato su un **quanto di tempo fisso** (*time slicing*).

Molto usato nei sistemi time-sharing. Se il quanto è troppo piccolo, cresce l'overhead; se è troppo grande, peggiora il tempo di risposta.

Shortest Process Next (SPN)

Seleziona il processo con il **minor tempo stimato di esecuzione**.

È senza prerilascio e può causare starvation dei processi lunghi. Stime possono essere fatte con medie aritmetiche o esponenziali.

Shortest Remaining Time (SRT)

Versione con prerilascio di SPN. Migliora il tempo di turnaround perché i job brevi hanno precedenza su quelli lunghi, ma richiede più overhead per stimare i tempi.

Highest Response Ratio Next (HRRN)

Seleziona il processo con il massimo valore di $R = \frac{w + s}{s}$ (**tempo di turnaround normalizzato**), favorendo i processi brevi senza causare starvation.

Confronto tra Strategie

- **FCFS**: semplice ma penalizza i processi brevi.
- **Round Robin**: bilanciato ma dipendente dal quanto.
- **SPN e SRT**: buone per tempi brevi ma rischiano starvation.
- **HRRN**: buon compromesso tra efficienza e fairness.

Schedulazione Fair-Share

Questa politica cerca di **distribuire equamente la CPU** tra gruppi di processi. Ogni gruppo ha una quota di CPU. Chi ha utilizzato più CPU rispetto alla quota viene penalizzato, chi ha utilizzato meno viene favorito.

Schedulazione UNIX tradizionale

Ottimizzata per il time-sharing, utilizza una strategia di feedback multilivello combinata con round robin ed ha un prerilascio dopo 1 secondo.

Le priorità si basano sul comportamento passato (uso CPU) e su un fattore controllato dall'utente chiamato "**nice**". Il sistema assicura che le diverse categorie di processi (swapper, I/O, utenti) restino separate per evitare ingiustizie.

Input/Output

Categorie di dispositivi I/O

- **Leggibili dall'uomo:** stampanti, terminali, mouse e tastiera.
- **Leggibili dalla macchina:** drive di dischi, USB, sensori.
- **Dispositivi di comunicazione:** modem.

Caratteristiche dei dispositivi I/O

1. **Velocità di trasferimento** con differenze tra un dispositivo ed un altro.
I più lenti sono mouse e tastiera per poi arrivare tra i più veloci come l'HDD, display grafico e la gigabit ethernet.
2. **Applicazioni:** l'uso di un dispositivo influenza il software.
3. **Complessità di controllo:** diversi dispositivi richiedono diversi software.
4. **Unità di trasferimento:** dispositivi a *caratteri* o a *blocchi*.
5. **Rappresentazione dei dati:** codifiche differenti.
6. **Condizioni di errore** diverse da un dispositivo all'altro.

Funzioni I/O

I/O Programmato

Il processo rimane in attesa attiva durante il completamento dell'operazione.

I/O guidato dalle interruzioni

Il processo può essere:

- **non bloccante** → il processore esegue istruzioni del processo che ha eseguito il comando di I/O.
- **bloccante** → il controllo passa al SO che sospende il processo corrente e ne schedula un altro.

DMA

Un modulo di DMA controlla lo scambio di dati tra memoria principale e dispositivo I/O.

Quando una periferica vuole trasferire dati, invia una **richiesta al DMA**. Il controller DMA prende il controllo del bus di sistema (senza coinvolgere la CPU), trasferisce i dati e **interrompe la CPU solo a fine operazione**.

Componenti principali

1. **Data Count (Conteggio dati):**
 - Tiene traccia del **numero di byte/parole da trasferire**.
 - Viene decrementato dopo ogni trasferimento completato.
2. **Data Register (Registro dati):**
 - Temporaneamente **immagazzina i dati** durante il trasferimento tra memoria e periferica I/O.

3. Address Register (Registro indirizzi):

- Contiene l'**indirizzo di memoria corrente** da cui leggere o scrivere.
- Viene aggiornato automaticamente ad ogni trasferimento.

4. Control Logic (Logica di controllo):

- Gestisce il **flusso operativo** del DMA.
- Decide quando iniziare/terminare il trasferimento, se leggere o scrivere, ecc.

Configurazioni di DMA

Single-bus, Detached DMA

- **Tutti i dispositivi (CPU, DMA, I/O, memoria) condividono lo stesso bus.**
- Il DMA è un **modulo separato** tra CPU e I/O.
- Caratteristiche:
 - **Semplice** da implementare.
 - Ma **bus condiviso = colli di bottiglia**: solo un componente alla volta può usare il bus

Single-bus, Integrated DMA-I/O

- Il DMA è **integrato nei controller delle periferiche I/O**.
- Gli I/O comunicano direttamente con la memoria.
- Caratteristiche:
 - Più efficiente rispetto al (a), perché **riduce la dipendenza dalla CPU**.
 - Resta comunque un **bus condiviso**, quindi c'è contesa.

I/O bus (architettura a due bus)

- Sistema con **due bus separati**:
 - **System bus**: CPU, memoria, DMA.
 - **I/O bus**: tutte le periferiche I/O.
- Il DMA fa da ponte tra i due bus.
- Caratteristiche:
 - **Maggiore parallelismo**: CPU e I/O possono lavorare in contemporanea.
 - Architettura più **scalabile** e **performante**.

Evoluzione della funzione I/O

1. Il processore controlla il dispositivo
2. viene aggiunto un controllore o modulo I/O
3. aggiunta delle interruzioni
4. DMA per controllo diretto sulla memoria
5. il modulo I/O diventa un processore separato con un suo instruction set (**canale di I/O**)

6. il modulo viene dotato di memoria locale e diventa un computer (**processore di I/O**).

Questo per coinvolgere meno il processore.

Obiettivi di progettazione

L'efficienza è il **principale obiettivo** nella progettazione dei meccanismi di input/output.

Le operazioni di I/O rappresentano spesso un **collo di bottiglia** per le prestazioni complessive del sistema.

I dispositivi di I/O sono **notevolmente più lenti** rispetto alla **memoria principale** e soprattutto rispetto al **processore**.

Questo problema è **particolarmente critico** nelle operazioni di I/O da/per disco, dove i tempi di accesso sono elevati.

L'obiettivo è fornire una **gestione uniforme dei dispositivi**, sia:

- nell'**interfaccia verso i processi** (trasparenza d'uso),
- sia nella **gestione interna da parte del sistema operativo**.

Tuttavia, questo obiettivo è **difficile da raggiungere** a causa della **grande eterogeneità tecnica** tra i dispositivi I/O (es. tastiere, dischi, stampanti, reti, ecc.).

Per superare questa difficoltà si adotta un **approccio gerarchico**:

- le **routine di basso livello** (driver) si occupano dei dettagli tecnici dei dispositivi,
- mentre i **livelli superiori** offrono un'interfaccia standard e astratta.

Struttura Logica

I/O Logico

Tratta il dispositivo come risorsa logica su cui si eseguono operazioni standard (`open()` , `close()` , `read()` , `write()`).

I/O sul dispositivo

Converte le operazioni richieste in sequenze di istruzioni.

Scheduling e controllo

- **Accodamento** delle richieste I/O in attesa,
- **Scheduling**: scelta dell'ordine con cui eseguire le operazioni (es. algoritmi per dischi come SSTF, SCAN, ecc.),
- **Controllo**: verifica dello stato del dispositivo,
- **Gestione delle interruzioni**: reagisce agli interrupt generati dal dispositivo quando un'operazione è completata.

Gestione di directory e file e organizzazione fisica

È responsabile della **gestione dell'astrazione dei file** e organizzazione fisica dei dati.

Buffering

Per evitare inefficienze o overhead nella gestione dell'I/O si eseguono **trasferimenti in input anticipati** e in **output in ritardo** rispetto alla richiesta.

Dispositivi a blocchi

Dischi rigidi, SSD, chiavette USB.

Caratteristiche:

- Dati organizzati in **blocchi di dimensione fissa** (es. 512B, 4KB).
- **Trasferimento avviene un blocco alla volta.**
- L'accesso avviene tramite il **numero del blocco** (accesso diretto o casuale).

Dispositivi a flusso (o a caratteri)

Terminali, stampanti, mouse, porte seriali, tastiere.

Caratteristiche:

- L'informazione è **non strutturata** in blocchi.
- I dati vengono trasferiti come **flusso continuo di byte** (o caratteri).

Buffer singolo (dispositivo a blocchi)

Quando un **processo richiede un'operazione di I/O**, il **sistema operativo (SO)** assegna un **buffer** nello **spazio del sistema**.

Durante un'operazione di input, i dati vengono **letti dal dispositivo e memorizzati nel buffer**.

- **Input anticipato (pre-fetching):**
 - Dopo che un blocco è stato trasferito nel buffer, viene **copiato nello spazio utente**.
 - Subito dopo, viene **richiesto un nuovo blocco** da caricare.
 - Questo consente al sistema di essere **pronto in anticipo** per successive richieste.

Vantaggi

- Il **processo può elaborare i dati del primo blocco**, mentre **il secondo blocco viene caricato in parallelo**.
- Il trasferimento avviene nello **spazio del sistema**, quindi il SO può:
 - **sospendere il processo**,
 - **scaricare la sua pagina dalla RAM (swapping)** senza perdere il dato in transito.

Svantaggi

- **Maggiore complessità** nella programmazione del SO:
 - Serve gestire correttamente lo stato del buffer e la sincronizzazione tra lettura/scrittura.
- **Influenza sulla gestione dello swapping:**
 - Anche se i dati stanno nel buffer del SO, bisogna **coordinare bene con la memoria virtuale** per evitare inconsistenze o duplicazioni.

Buffer singolo (dispositivo a flusso)

Una linea alla volta

Terminali scroll, stampanti a linee.

Input diviso in linee che terminano con "invio"

- Processore sospeso in attesa della linea di input

Un byte alla volta

Terminali in modo 'form', sensori, controller.

- La line di output è trasferita nel buffer ed il processo continua.

Doppio buffer

Un processo trasferisce dati da/a un buffer mentre il SO svuota o riempie l'altro (**buffer swapping**).

Buffer circolare

Più di due buffer caricati e scaricati **circolarmente**.

Parametri di Prestazione del Disco

Le prestazioni dell'I/O su disco dipendono da:

- Sistema di elaborazione
- Sistema operativo
- Canale I/O e controller del disco

Il disco ruota a **velocità costante**.

I dati sono distribuiti in **tracce** e **settori**.

Accesso ai dati

La **testina** deve:

- Posizionarsi sulla **traccia** (movimento fisico).
- Attendere il **settore desiderato** (rotazione del disco).

Tempi principali

- **Tempo di ricerca**: per raggiungere la traccia.
- **Ritardo rotazionale**: attesa che il settore passi sotto la testina.
- **Tempo di accesso** = *Tempo di ricerca* + *Ritardo rotazionale*.

Tipi di dischi

- **Teste fisse**: una testina per traccia → accesso più veloce.
- **Teste mobili**: una testina si muove tra le tracce → richiede tempo di ricerca.

Tempo di accesso

Tempo di ricerca

Tempo necessario a muovere il braccio del disco sulla traccia richiesta.

- Tempo di avvio
- Tempo necessario per attraversare i cilindri (comprensivo di **settling time** → affinché l'uscita raggiunga e rimanga entro un certo margine di errore)

Più piccolo è il disco, minore è il tempo di ricerca (oggi $< 10\ ms$)

Ritardo rotazionale

a 3600 rpm, in media 8,3s

a 15000 rpm, in media 2s

Tempo di trasferimento

$$T = \frac{b}{rN}$$

b = bit da trasferire

N = bit per traccia

r = velocità di rotazione (rps)

Tempo di accesso medio

$$T_a = T_s + \frac{1}{2r} + \frac{b}{rN}$$

b = numero di bit da trasferire,

N = numero di bit per traccia,

r = velocità di rotazione in rps,

T_s = tempo di ricerca medio

Politiche di schedulazione del disco

Per ridurre il tempo di ricerca si possono adottare diverse politiche:

1. **FIFO**: richieste processate in ordine di arrivo e, con molti processi, approssima la schedulazione random.
2. **Priorità**: politica al di fuori del software di controllo del disco in cui si dà priorità a processi interattivi, penalizzando i lavori batch.
3. **SSTF (Shortest Service Time First)**: precedenza alla richiesta con tempo di servizio minore (possibile starvation).
4. **SCAN**: il braccio si muove sempre in una direzione soddisfacendo tutte le richieste su ciascuna traccia, alla fine inverte la direzione.
5. **C-SCAN**: scansione in un'unica direzione, raggiunta l'ultima traccia il braccio torna alla prima e ricomincia.
6. **N-Step-SCAN**: richieste inserite in sotto-code (di lunghezza N), processate una alla volta con SCAN; nuove richieste non possono essere inserite nella coda attualmente processata.
7. **FSCAN**: due sotto-code, una tenuta vuota inizialmente e riempita mano a mano con le nuove richieste.

Gestione dei File

File System

Il file system è uno strumento per **memorizzare dati organizzati in file**.

Ha una collezione di **funzioni** (*crea, cancella, apri, chiudi, leggi, scrivi*) per operare sui file e gestisce gli **attributi** associati ad esso.

Sistema di gestione dei file

Fornisce servizi agli utenti ed alle applicazioni che permettono l'utilizzo dei file, questo per:

- garantire validità dei dati
- ottimizzare le prestazioni
- supporto I/O per i vari tipi di dispositivi di memorizzazione
- minimizzare rischio di perdita dati
- fornisce procedure I/O con interfaccia standard
- supporto I/O per più utenti.

Esigenze degli utenti

- creare, cancellare, leggere, scrivere e modificare i file
- accesso controllato a file di altri utenti
- controllo su chi può accedere a file di altri utenti
- muovere dati tra i file
- backup e ripristino in caso di danneggiamento
- accesso ai file per nome, anziché per ID.

Architettura del sistema di gestione dei file

- **User Program:** programma dell'utente che accede ai file attraverso chiamate di alto livello (`open`, `read`, `write`, `close` ...).

1. Metodi di accesso ai file (file organization)

- **Pile:** dati scritti senza ordine; semplice ma inefficiente per ricerche.
- **Sequential:** dati ordinati e letti in sequenza.
- **Indexed Sequential:** combina indice e accesso sequenziale.
- **Indexed:** utilizza solo un indice per accedere direttamente ai dati.
- **Hashed:** usa una funzione di hash per accesso rapido.

2. Logical I/O

- Fornisce un'interfaccia logica ai file (gestione buffer, record, ecc.).
- Nasconde i dettagli fisici all'utente.

3. Basic I/O Supervisor

- Coordina le operazioni di I/O.
- Gestisce il traffico tra processi e periferiche.
- Si occupa delle **interruzioni**.

4. Basic File System

- Gestisce la lettura/scrittura dei blocchi fisici sul disco.
- Conosce la struttura fisica del file system (blocchi, settori, tracce).

5. Device Driver

- **Disk Device Driver / Tape Device Driver**
- Traduce i comandi generici in istruzioni specifiche per il dispositivo.

Driver dei dispositivi

Software di basso livello che comunicano direttamente con i dispositivi periferici, avviando e gestendo le operazioni di I/O.

Base del file system

Fornisce un'**interfaccia logica ai file** (gestione buffer, record, ecc.) e si occupa dei blocchi di **dati scambiati** con il disco, del **posizionamento** dei blocchi in memoria secondaria ed organizza i buffer in memoria principale.

Non conosce né il contenuto né la struttura del file.

Supervisore dell'I/O base

Responsabile di **inizio e terminazione** di tutte le operazioni di I/O su file.

Seleziona il dispositivo su cui eseguire I/O e si occupa della schedulazione del disco e degli altri dispositivi per ottimizzare le prestazioni,

Assegna i buffer I/O e alloca la memoria secondaria.

I/O Logico

Realizza le basi di dati relative ai file, abilitando utenti e applicazioni ad accedere ai record e fornendo le funzioni di I/O ai record.

Metodi di accesso

Fornisce un'interfaccia standard tra le applicazioni e il file system e i dispositivi.

Directory

- Ricerca
- Crea file
- Cancella file
- Listare contenuto dir
- Aggiornare la dir

Schema a due livelli

Una directory principale contiene una entry per ogni directory utente con indirizzo e dati di accesso. Ogni directory utente è una lista di file suoi con nomi unici solo all'interno di ciascuna directory ed è uno **schema ad albero** che può contenere sottodirectory.

Organizzazione dei file

File a mucchio

Dati in ordine di arrivo, con record aventi campi differenti a cui si accede con ricerca esaustiva.

File sequenziali

Record in formato fisso, identificati da un campo **chiave**, utili per app batch lente.

Le modifiche avvengono tramite un **file di transazioni**.

File sequenziale indicizzato

Aggiunge un indice per **accesso random**, accelerando l'accesso ai record.

Le modifiche (nuovi record) non riscrivono tutto il file:

- Vengono memorizzate in un **file di overflow**.
 - Accessibili tramite **puntatori** dal file indicizzato.
-

File indicizzato

Record accessibili attraverso un **indice totale** e un **indice per ogni campo**, aggiornati ad ogni modifica.

I record possono essere a **lunghezza variabile** e sono di veloce accesso.

File diretto (ad hash)

Hashing della chiave per accedere direttamente al blocco del record.

Nessuna organizzazione sequenziale → l'ordine fisico dei record è irrilevante.

Allocazione dei file

Un **file** è composto da una **collezione di blocchi** su memoria secondaria (es. disco).

Il **Sistema Operativo** si occupa dell'**allocazione dei blocchi** ai file.

La **strategia di allocazione** adottata influisce anche sulla **gestione dello spazio libero**.

Lo spazio viene allocato in **porzioni** → insiemi di **blocchi contigui** o meno.

Per tenere traccia dei blocchi allocati a ciascun file si usa una struttura dati chiamata **FAT** (*File Allocation Table*).

Strategie di Allocazione

Preallocazione

- Richiede di **dichiarare la dimensione del file** in fase di creazione.

- Problemi:
 - Spesso difficile stimare correttamente.
 - Si tende a **sovrastimare**, sprecando spazio.

Allocazione Dinamica

- Lo spazio viene allocato **man mano che il file cresce**.
- Più flessibile ed efficiente.
- Può causare **frammentazione**.

Dimensione delle porzioni

Esigenze in contrasto

- **Contiguità**: migliora le prestazioni (soprattutto nei SO per transazioni).
- **Tante porzioni piccole**: aumentano la **dimensione delle tabelle** di allocazione.

Porzioni contigue grandi e variabili

- **Prestazioni efficienti** (più dati contigui).
 - **Riduzione degli sprechi** (miglior uso dello spazio).
 - **Tabelle di allocazione più piccole**.
-
- Meno flessibili nella gestione dinamica.

Blocchi

- **Maggiore flessibilità**.
 - Blocchi **allocati al bisogno**.
-
- Richiede **tabelle di allocazione più grandi**.
 - Non garantisce la **contiguità**.

Allocazione contigua

Ogni file è memorizzato in un insieme di **blocchi contigui** sul disco.

La FAT del file contiene:

- L'**indirizzo del primo blocco**.
- La **lunghezza del file** (numero di blocchi).

✓ Vantaggi

- **Accesso sequenziale** efficiente: La testina del disco non deve spostarsi tra blocchi.
- **Accesso diretto** semplice: Calcolo immediato della posizione del blocco desiderato.
- **Gestione semplice**: Struttura dati minimale per tracciare i blocchi del file.

✗ Svantaggi

- **Frammentazione esterna**: Difficoltà nel trovare spazio contiguo per nuovi file o per l'espansione di file esistenti.
- **Difficoltà nell'espansione**: Se il file cresce oltre lo spazio allocato, potrebbe essere necessario spostarlo.
- **Necessità di preallocazione**.

Allocazione contigua (dopo compattazione)

La **compattazione** è un processo che riorganizza i file sul disco per **eliminare la frammentazione esterna**, consolidando lo spazio libero in un'unica area contigua.

Il sistema operativo **sposta i file** per creare una grande area di spazio libero contiguo.

Questo permette di allocare nuovi file o espandere quelli esistenti senza frammentazione

Allocazione a catena

Ogni file è memorizzato come una **lista concatenata di blocchi** sul disco.

Ogni blocco contiene:

- I **dati** del file.
- Un **puntatore al blocco successivo** del file.

Il **primo blocco** è indicato nella directory del file; l'ultimo blocco contiene un **indicatore di fine** (es. null o -1).

✓ Vantaggi

- **Nessuna frammentazione esterna:** i blocchi possono essere allocati ovunque sul disco.
- **Facilità di crescita:** è semplice aggiungere nuovi blocchi al file.
- **Semplicità nella gestione dello spazio libero:** non è necessario trovare blocchi contigui.

✗ Svantaggi

- **Accesso diretto non supportato:** per accedere a un blocco specifico, è necessario attraversare la catena dall'inizio.
- **Overhead di memorizzazione:** ogni blocco deve contenere un puntatore, riducendo lo spazio disponibile per i dati.
- **Affidabilità:** la perdita o la corruzione di un puntatore può rendere inaccessibile il resto del file.

Allocazione a catena (dopo compattazione)

Sebbene l'allocazione a catena eviti la frammentazione esterna, nel tempo i blocchi di un file possono diventare **dispersi** sul disco.

Dopo la compattazione i blocchi del file vengono allocati contiguamente in ordine.

Allocazione indicizzata con porzioni a blocchi

Ogni file è associato a un **blocco indice** che contiene i puntatori ai blocchi dati del file.

I dati del file sono memorizzati in **blocchi** (porzioni) di dimensione fissa, che possono essere non contigui sul disco.

✓ Vantaggi

- **Supporto per accesso sequenziale e diretto.**
- **Eliminazione della frammentazione esterna.**
- **Facilità nella gestione dello spazio libero.**

✗ Svantaggi

- **Overhead di memorizzazione:** il blocco indice occupa spazio aggiuntivo.
 - **Limiti nella dimensione del file:** se il blocco indice è di dimensione fissa, può limitare la dimensione massima del file; l'uso di indici multilivello aumenta la complessità.
-

Allocazione indicizzata con porzioni a lunghezza variabile

In questo schema, ogni file è associato a un **blocco indice** che contiene il **numero del blocco di inizio** e la sua **lunghezza**.

✓ Vantaggi

- **Riduzione della frammentazione interna:** le porzioni si adattano meglio alla dimensione effettiva dei dati.
- **Prestazioni migliorate:** l'accesso diretto tramite l'indice consente tempi di accesso più rapidi rispetto all'allocazione concatenata.
- **Scalabilità:** adatto per file di dimensioni variabili e grandi dimensioni.

✗ Svantaggi

- **Complessità nella gestione dello spazio libero:** la variabilità delle dimensioni delle porzioni complica il tracciamento e l'allocazione dello spazio disponibile.
 - **Overhead di memorizzazione:** il blocco indice occupa spazio aggiuntivo e deve essere gestito con attenzione.
 - **Possibile frammentazione esterna:** se non gestita correttamente, la variabilità delle porzioni può portare a frammentazione dello spazio su disco.
-

Gestione degli spazi liberi

La **tabella di allocazione del disco** tiene traccia dei blocchi disponibili.

Bitmap

Un vettore con **un bit per ogni blocco** (0 libero, 1 allocato).

Porzioni libere concatenate

Ogni porzione libera contiene la **lunghezza** ed un **puntatore alla porzione libera successiva**, che viene aggiornato ad ogni allocazione o rilascio.

Indicizzazione

Anche lo spazio libero ha la sua FAT con indice basato su porzioni a lunghezza variabile.

Gestione file in UNIX

1. **Ordinari:** contiene dati arbitrari in 0 o più blocchi.
2. **Directory:** contiene una lista di filename con i puntatori ai relativi **i-node**.
3. **Speciali:** usati per accedere a dispositivi periferici.
4. **Pipe con nome:** per la comunicazione tra processi.

5. **Link:** nome alternativo per un file esistente.
6. **Link simbolico:** collegamento ad un altro file.

I-node

Tutti i file in UNIX sono amministrati dal SO attraverso gli **i-node** (indexed node).

Contiene le informazioni chiave di cui il sistema ha bisogno per un file.

Più nomi di file possono essere associati ad un singolo i-node, ma un i-node è associato ad un unico file e ogni file è controllato da un unico i-node.

Allocazione dei file

- **Allocazione a blocchi**
- **Allocazione dinamica**
- **Indicizzazione** mediante puntatori diretti (nell'i-node) e tre puntatori indiretti (singoli, doppi, tripli).

Struttura del volume

Il file system UNIX risiede su un singolo disco, contenente:

1. **Boot block:** codice per avviare il sistema.
 2. **Superblock:** informazioni del file system.
 3. **Tabella degli i-node.**
 4. **Blocchi dati:** spazio disponibile per i file dati e le directory.
-